

Standup Page

Introduction

The Team Standup Report enhances your standup meetings with valuable data and insights. It highlights your in-progress pull requests and tickets, along with recently completed work. This helps your team stay aligned, identify priorities, and address blockers efficiently. Use these insights to keep work moving, remove obstacles, and free up bandwidth for meaningful contributions.

Monitoring the Sprint

it is always helpful to track the development team's progress toward completing the work of a single iteration because it leads to tracking progress toward the high level goal of a release.

- **The Task Board:** Provides a visual representation of work items, tracking tasks from "To Do" to "Done." It helps teams stay organized, identify bottlenecks, and adjust priorities as needed.

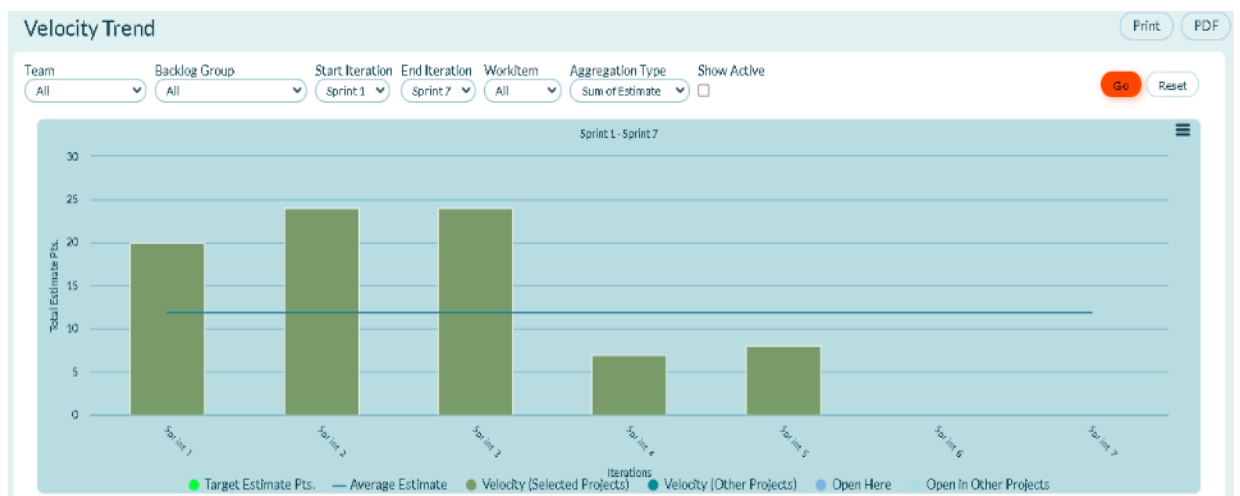
- **Iteration Burndown Charts:** Display remaining work over time, offering insights into whether the team is on track to complete the iteration. A steadily declining burndown line indicates smooth progress, while flat or erratic trends signal potential issues.
- **Tracking Effort Expended:** Helps measure the amount of effort spent on tasks, ensuring resources are allocated efficiently. Comparing actual effort against estimates enables teams to refine their planning accuracy.
- **Individual Velocity:** Tracks each team member's work output over iterations, aiding in workload distribution and future sprint planning. Understanding individual velocity helps optimize team performance and balance assignments effectively.

Velocity

Team

Velocity tells us when something can be delivered. It represents the predictability of the team's estimates and ability to complete work from sprint to sprint. Stability in this metric drives Trust.

The **Velocity Trend** report shows a team's historical velocity over a set period of time. This report keys off the story-level estimate and averages what the team has actually been able to deliver per sprint or iteration



This image shows a **Velocity Trend Report** used in Agile project management. The chart displays the team's historical velocity across multiple sprints, helping track delivery consistency, estimate predictability, and sprint performance over time.

Velocity Trend

Individual (Not recommended)

Velocity is typically used for teams, but it can also be calculated for an individual by tracking their average story points completed per sprint.

Formula

$$\text{Individual Velocity} = \frac{\sum \text{Story Points Completed Over Multiple Sprints}}{\text{Number of Sprints}}$$

Description for the formula = This image explains the formula used to calculate **Individual Velocity** in Agile. It shows that velocity is calculated by dividing the total story points completed across multiple sprints by the number of sprints.

Sprint	Story Points Completed
Sprint 1	18
Sprint 2	22
Sprint 3	16
Sprint 4	20

Alice's Velocity Calculation:

$$\frac{18 + 22 + 16 + 20}{4} = \frac{76}{4} = 19 \text{ (Avg. Story Points per Sprint)}$$

Description = This image provides an example of calculating an individual's velocity using sprint data. Alice completed 18, 22, 16, and 20 story points across four sprints, resulting in an average velocity of **19 story points per sprint**.

Story Churn

Team

Teams use this metric to help improve their ability to focus and achieve the goals originally set forth in Sprint Planning.

Story Churn, also known as the "Chaos Metric", is the number of stories/defects added to or removed from an iteration over time. If the number is high, it's a strong indicator the team is frequently context switching. The goal should be a low, steady downward trend.

High churn can indicate **scope creep**, unclear requirements, or shifting priorities.

Low churn suggests **stable planning** and better backlog refinement.

$$\text{Story Churn \%} = \left(\frac{\text{Changed Stories (Added + Removed + Modified)}}{\text{Total Stories in Sprint}} \times 100 \right)$$

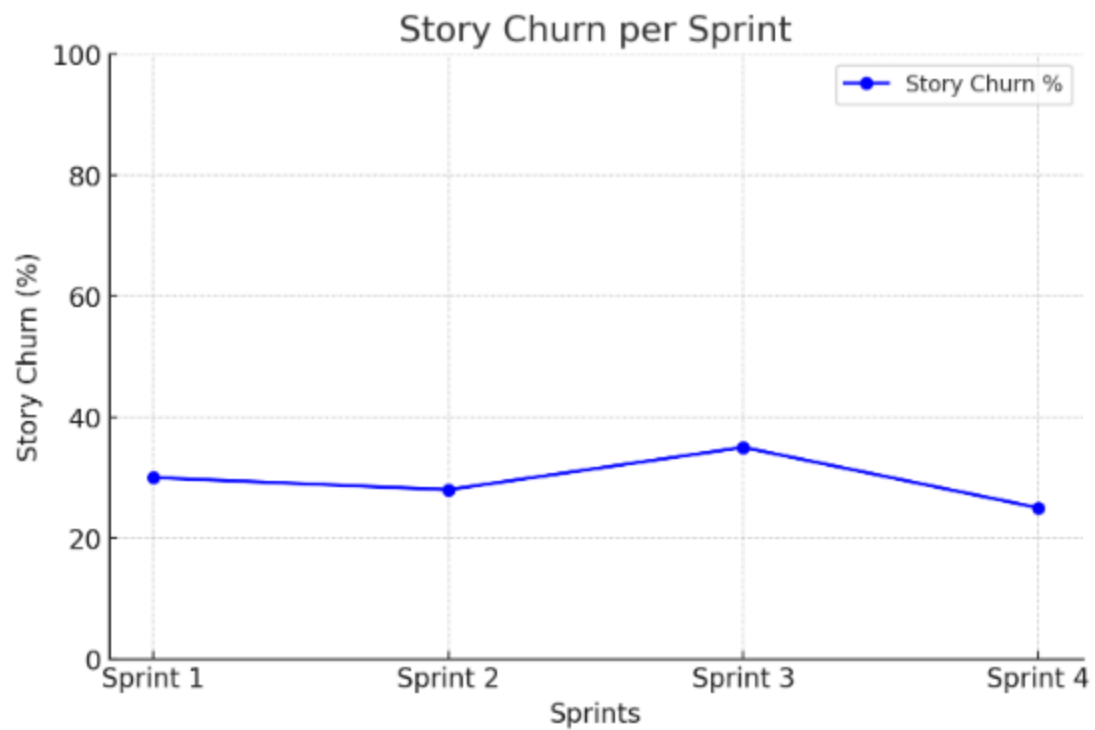
Formula

Description = **Story Churn Formula** The formula for calculating Story Churn %: the sum of Changed Stories (Added + Removed + Modified) divided by Total Stories in Sprint, multiplied by 100.

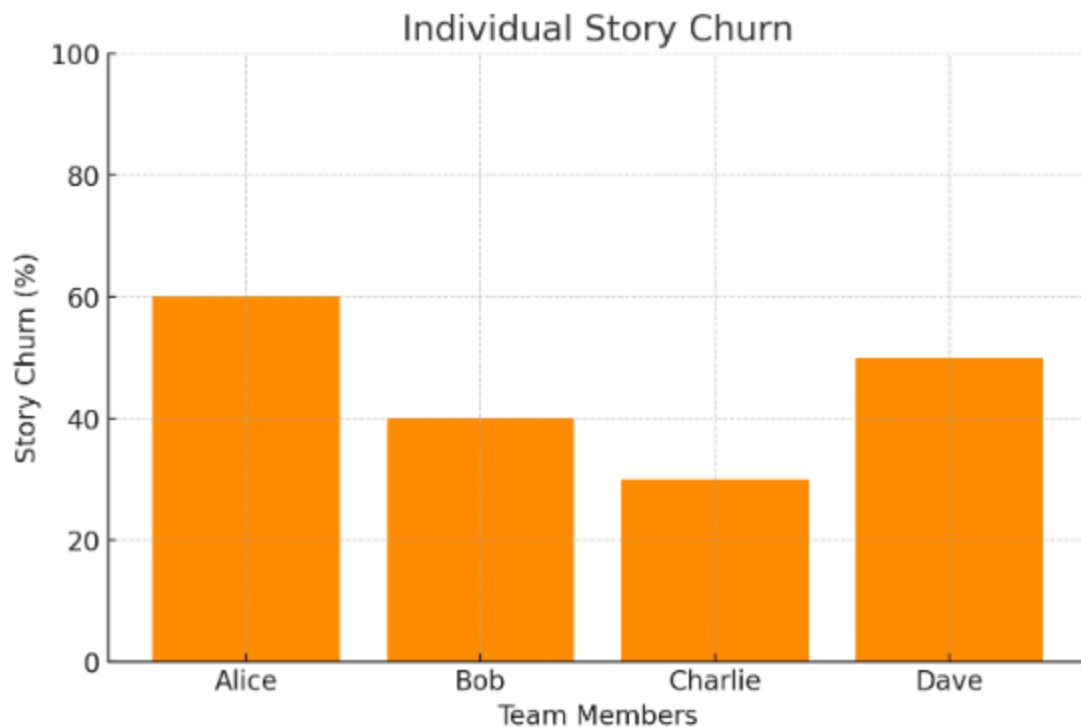
Example Calculation:

Sprint	Stories Planned	Added	Removed	Modified	Story Churn %
Sprint 1	20	3	2	1	$\frac{(3+2+1)}{20} \times 100 = 30\%$
Sprint 2	18	2	1	2	$\frac{(2+1+2)}{18} \times 100 = 28\%$

Description = **Example Calculation Table** A table demonstrating how Story Churn % is calculated across two sprints. Sprint 1 had 20 planned stories with 3 added, 2 removed, and 1 modified, resulting in a 30% churn. Sprint 2 had 18 planned stories with 2 added, 1 removed, and 2 modified, giving a 28% churn.



Description = **Story Churn per Sprint (Line Chart)** A line chart tracking team-level Story Churn % across 4 sprints. Churn starts at ~30% in Sprint 1, dips slightly in Sprint 2, peaks at ~35% in Sprint 3, and drops to ~25% in Sprint 4, showing some variability but no strong downward trend yet.



– **Individual Story Churn (Bar Chart)** A bar chart comparing Story Churn % across four team members. Alice has the highest churn (~60%), followed by Dave (~50%), Bob (~40%), and Charlie (~30%), indicating Alice and Dave face the most workload instability.

Individual

Individual Story Churn measures how much an individual's assigned stories are **added, removed, or modified** during a sprint. It helps assess how stable their workload is and whether constant changes are affecting productivity.

Identifies if **one person is facing too much scope change**

Helps in **reducing interruptions and stabilizing work**

Ensures **better planning for individual workloads**

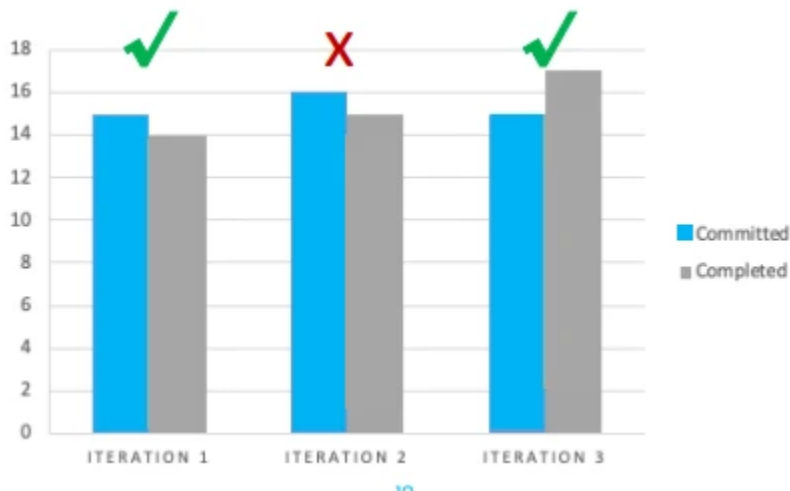
Higher churn % means more **added/removed/modified stories**, indicating instability.

Lower churn % suggests more **consistent workload**.

Sprint Goal Success Rate

This metrics acknowledges that teams perform “better” when working towards a common goal. A Sprint Goal can also help break down objectives into smaller, easier-to-deliver outcomes.

Sprint Goal Success Rate measures the percentage of time a team successfully reaches its sprint goal in a given period.



Description = A bar chart showing Committed vs. Completed stories across 3 iterations. Iterations 1 and 3 met their sprint goals, while Iteration 2 fell short, resulting in a 66% Sprint Goal Success Rate.

Burn Down

Team

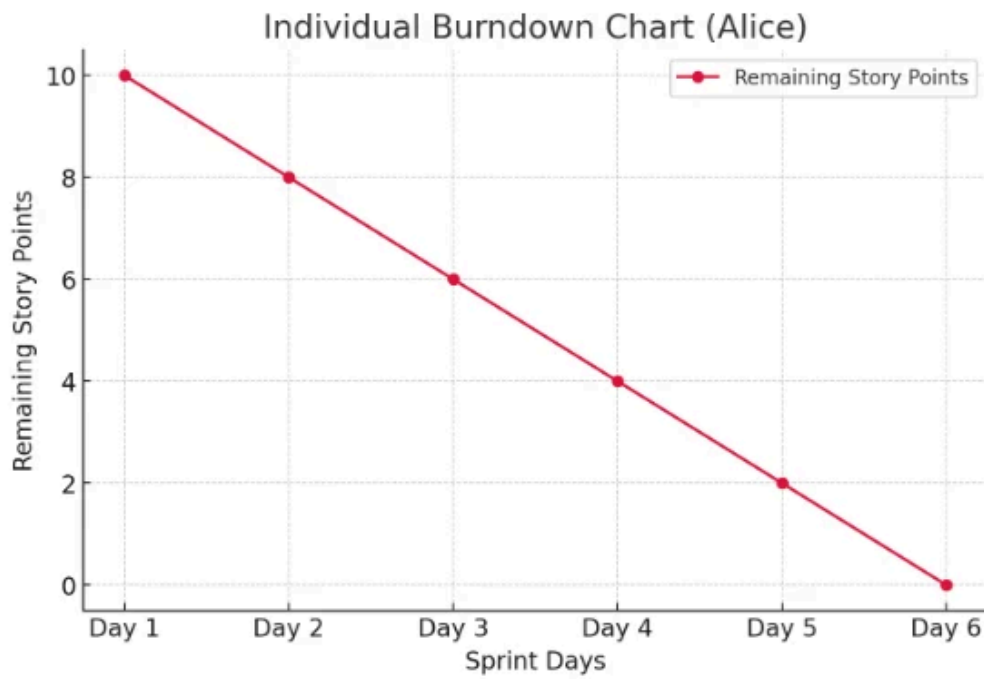
Display remaining work over time, offering insights into whether the team is on track to complete the iteration. A steadily declining burndown line indicates smooth progress, while flat or erratic trends signal potential issues.

Individual

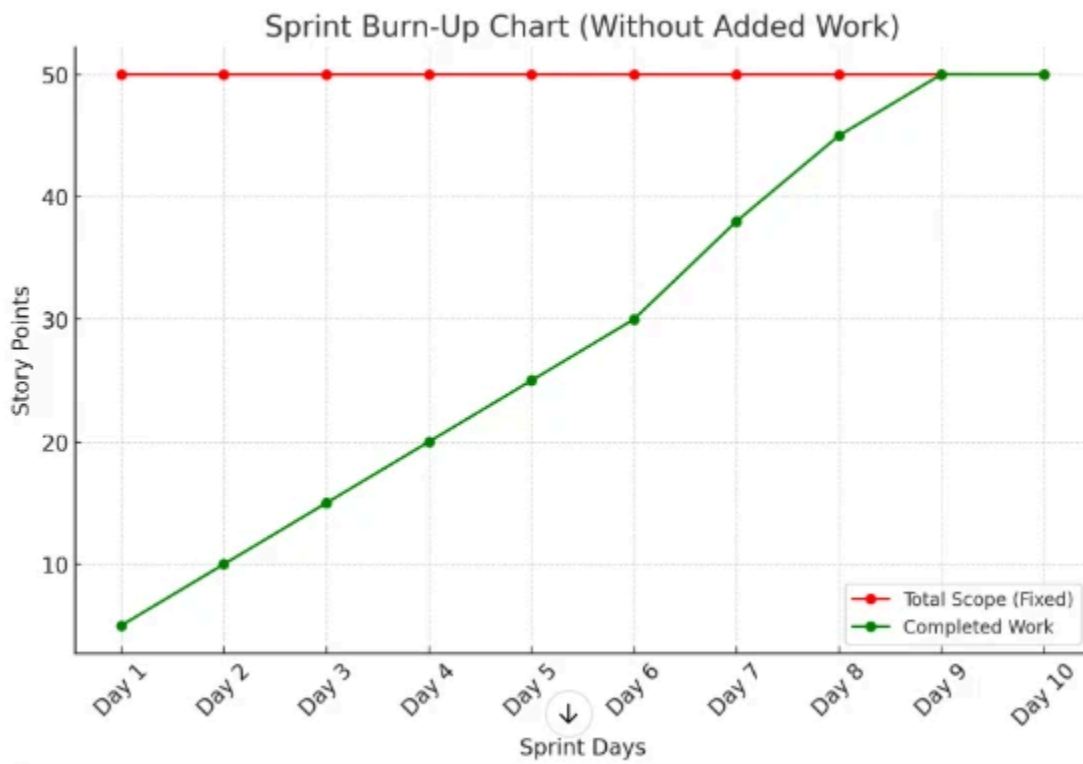
Each individual starts with a **total assigned story points** at the beginning of the sprint.

- 1.Track how many story points they **complete** each day.
- 2.Subtract from the total to get **remaining work**.

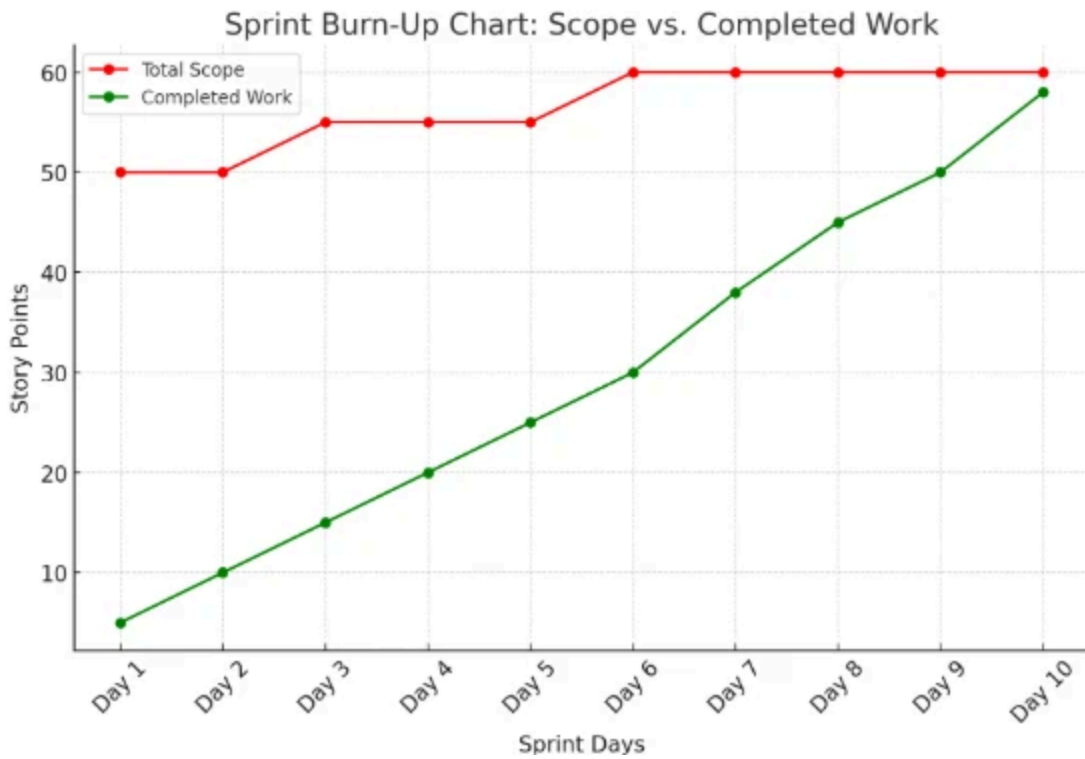
Day	Remaining Story Points (Alice)
Day 1	10
Day 2	8
Day 3	6
Day 4	4
Day 5	2
Day 6	0



With out Added Work Mid Sprint.



With Added Work Mid Sprint.



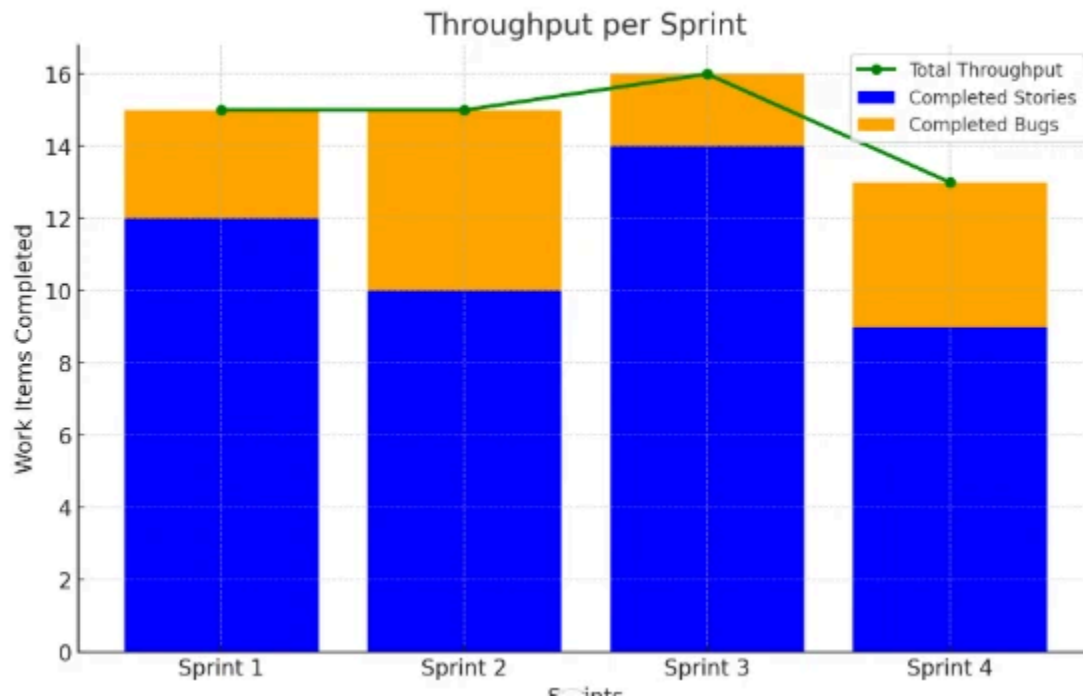
Throughput

Tracks the number of user stories or features delivered over time. Helps assess the team's ability to consistently deliver value to stakeholders.

Throughput is the measure of how many days it takes to process a request, from when development starts until the team's DoD.

$$\text{Throughput} = \frac{\text{Total Completed Work Items}}{\text{Time Period (Sprint)}}$$

Sprint	Completed Stories	Completed Bugs	Total Throughput
Sprint 1	12	3	15
Sprint 2	10	5	15
Sprint 3	14	2	16
Sprint 4	9	4	13



Test Coverage Metrics

Statement Coverage: Measures the percentage of code statements executed by automated tests. While valuable, it doesn't guarantee thorough testing of all code paths.

Branch Coverage: Extends statement coverage by measuring the percentage of decision points (branches) in the code that are exercised by tests. Provides more comprehensive insight into code coverage.

Path Coverage: Goes even deeper, measuring the percentage of code paths exercised by tests. Offers the most thorough assessment of code coverage but can be resource-intensive to achieve.

Defect Metrics

Defect Density: Calculates the number of defects identified per unit of code or functionality. Helps identify areas of the application with higher defect rates, guiding testing focus.

- **Defect Reopen Rate:** Measures the percentage of defects that are reopened after being marked as resolved. Indicates the effectiveness of defect resolution processes and the stability of fixes.
- **Time-to-Defect Resolution:** Tracks the average time taken to resolve defects from identification to closure. A lower time-to-resolution indicates efficient defect management and faster feedback loops.

Blocker & Dependency Metrics

Blocked Tasks Count – How many tasks are currently blocked?

Dependency Issues – Are there external dependencies delaying progress?

Sprint Progress Metrics

Story Points Completed vs. Remaining – Tracks work done vs. pending in the sprint.

Planned vs. Actual Work Completed – Identifies scope creep or delays.

Team capacity and workload distribution

Capacity Planning Table

Formula for Workload %:

$$\text{Workload \%} = \left(\frac{\text{Story Points Allocated}}{\text{Team Member's Estimated Capacity in Story Points}} \times 100 \right)$$

Team Member	Estimated Capacity (Story Points)	Story Points Allocated	Workload %
Alice	20	18	90%
Bob	22	15	68%
Charlie	18	16	89%
Dave	25	20	80%

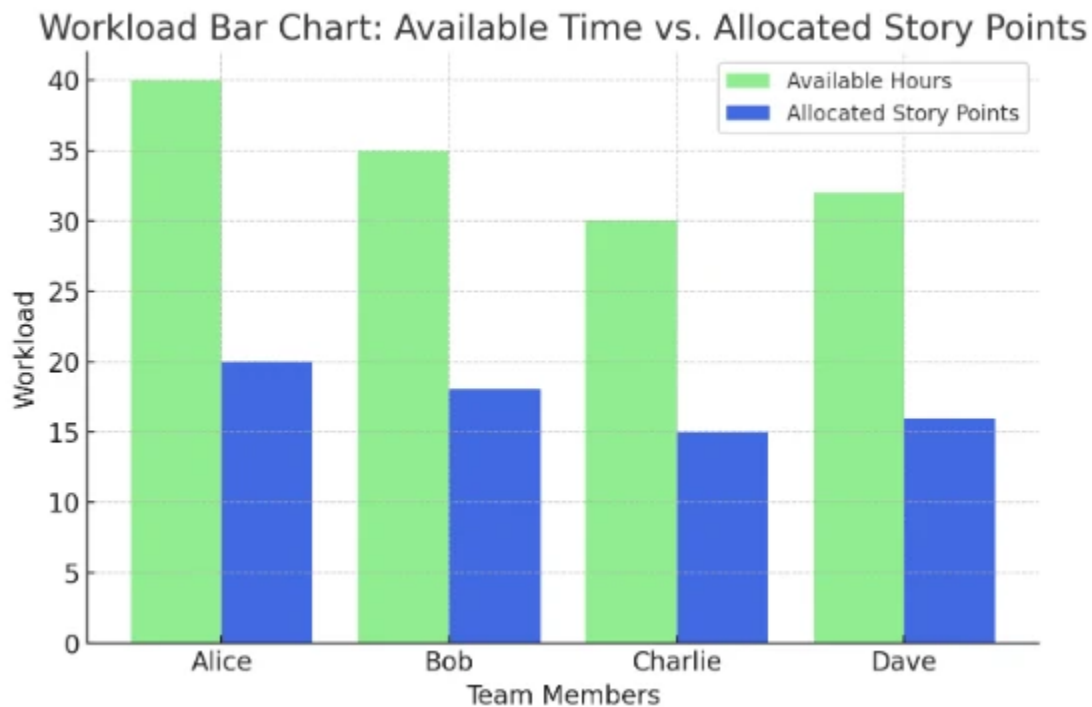
Story Points - Relative Estimation

$$\text{Workload} = \frac{\sum \text{Assigned Story Points}}{\text{Available Capacity (Ideal Hours/Days)}}$$

Team Member	Available Days (Sprint)	Story Points (Ideal Days) Allocated	Workload %
Alice	10	8	80%
Bob	10	9	90%
Charlie	8	7	87.5%
Dave	9	6	67%

Story Points - Ideal Days

Workload Bar chart



Defect Leakage Analysis

Defect Leakage Analysis is a metric to identify the efficiency of Testing. It measures the percentage of bugs leaked from the current testing phase to the Production. This Metric will be calculated as an average % for the entire Engineering and reported in the Summary tab of the Weekly Engineering Operational Metrics, as well as per Portfolio/Product.

Metric Formula:

$$\text{DLA} = \left(\frac{\text{No. of bugs found in Production per Product}}{\text{No. of bugs found in Testing per Product} + \text{No. of bugs found in Production per Product}} \right) \times 100$$

DLA Metric is fully automated and calculated based on Jira Queries

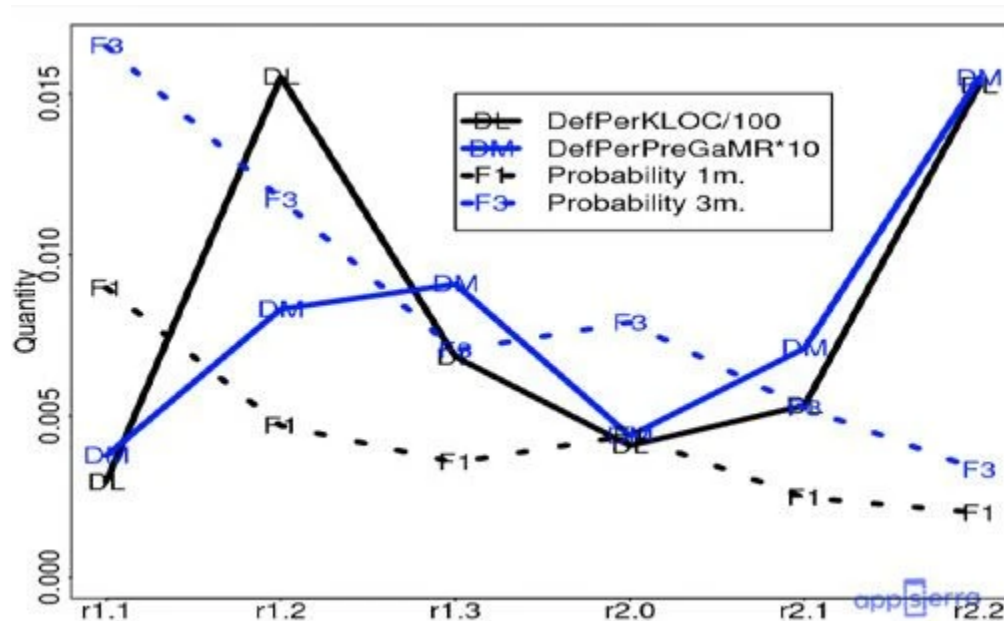
Link to Jira Story: <https://trigentnav.atlassian.net/browse/QM-1277>

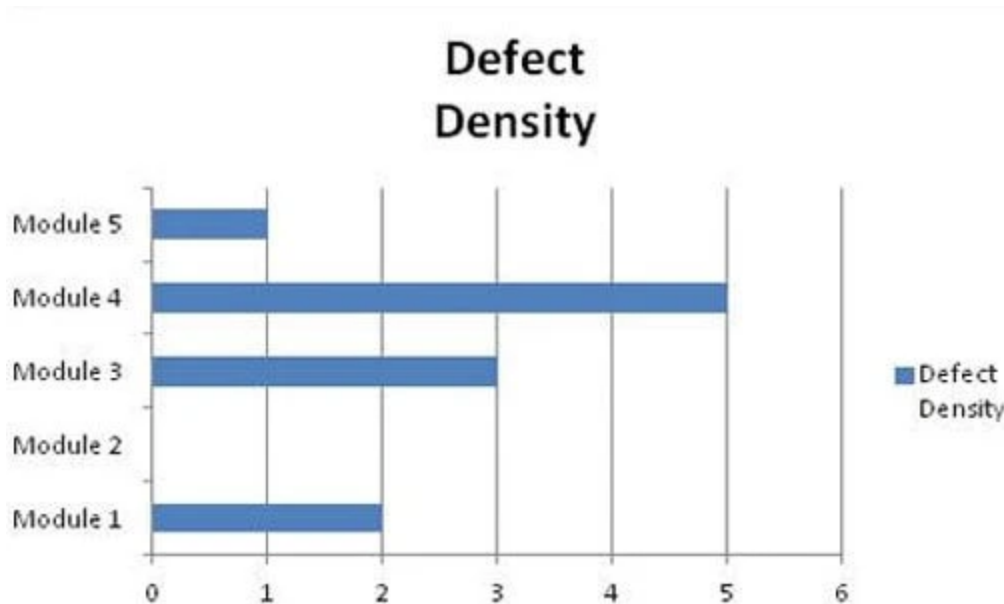
Defect Density

Defect Density measures the number of defects in a software component per unit of size. It helps teams identify areas for improvement and make informed decisions to enhance overall software reliability and user satisfaction. Defect Density can also be termed as the ratio of the number of defects and the size of the software.

Defect density is important for Agile teams as it can help them detect and fix defects early, thereby reducing the cost and time of fixing errors. It can also be used to improve code quality and maintainability, while measuring and demonstrating the value they deliver to customers.

Defect density helps in predicting the number of defects that may exist in the future development of the software. It also helps in analyzing the efficiency of the testing process in detecting defects and the amount of testing necessary to undertake.





Link to the Jira story: <https://trigntnav.atlassian.net/browse/QM-112>

Defect Rejection Ratio

Defect rejection means the defects that are rejected by the project team. It may be due to the 3 main reasons: Irreproducible, Incorrect and Duplicate bugs.

Defect rejection ratio refers to the number of defects that were raised that have been rejected divided by the total number of defects raised.

Description of Total Defects Rejected -

- **Description:** Total number of rejected defects refers to the number of defects which are rejected so far
- **Formula:** Total Defects Rejected = Total number of all rejected defects

Purpose:

This provides details to measure the effectiveness of a testing team



Link to the Jira story: <https://trigntnav.atlassian.net/browse/QM-68>

PR Raised by Individual.



By Mohan_ra

1 min

See views

Add a reaction

Pull Requests (PRs) are an essential part of Agile teams using **Git-based workflows**. Including PR discussions in **daily standups** helps teams track code progress, unblock reviews, and maintain development velocity.

Ways to Use PRs in Standups

- Track PR Progress in the "Yesterday-Today-Blockers" Format

✅ **Yesterday:** "I opened PR#101 for the authentication module."

🔥 **Today:** "I'll address review feedback and merge it."

🚧 **Blocker:** "Waiting for review from @John."

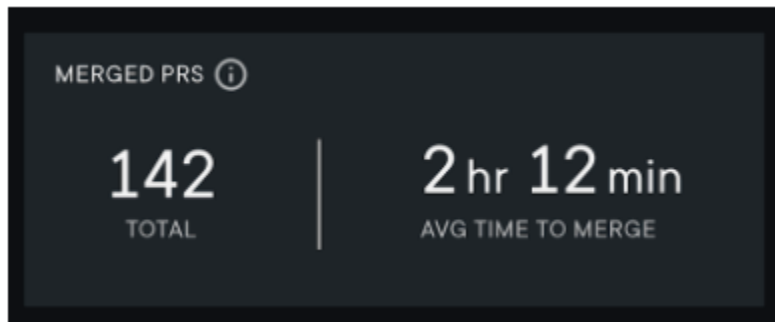
- Use the "In Progress" Tab & Bar Chart View
 - Highlight **PRs that have been open the longest**.
 - Identify **stalled PRs** that need attention.
 - Encourage team members to **prioritize code reviews**.
- Assign Code Reviews as Action Items
 - If a PR is waiting on a review, assign it directly in the standup.
 - Example: "@Lisa, can you review PR#202 today?"
- Identify PR Bottlenecks & Dependencies
 - If a feature depends on a PR being merged, the team can plan accordingly.
 - Example: "The API changes in PR#150 must be merged before I start integration."

Open PR's

- List of PR's Pending Reviews
- List of PR's Reviews completed
- Total PR which are open.

Merged PR's

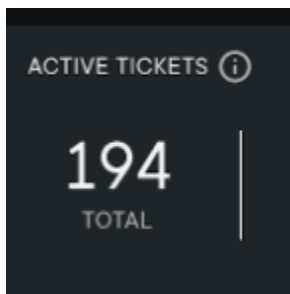
- Total PR Merged which is raised by the Individual team member.
- Average time to Merge



Individual Ticket Status

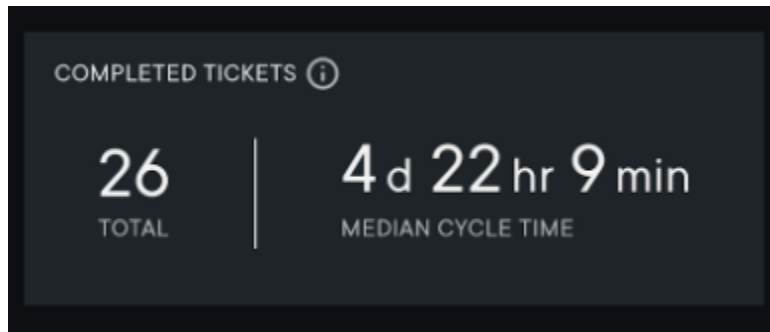
Active Tickets:

Active Tickets Section Displays ticket which is assigned to the individual team member which is not in done status.



Completed Tickets:

Completed Ticket card depict tickets completed by the Individual and the Median cycle time.



Estimated vs. Actual Time Spent

To check if work is taking longer than expected.

Sprint Commitment Metrics.



Story Points Completed – How many points have you finished?



Story Points Remaining – How much is left to be done?

Blocker Metrics



Blocked Tasks – What is preventing progress?



Dependency Issues – Are you waiting for another task/PR to be completed?



Time Since Blocked – How long has the blocker been unresolved?

Burndown

Display remaining work over time, offering insights into whether the team is on track to complete the iteration. A steadily declining burndown line indicates smooth progress, while flat or erratic trends signal potential issues.

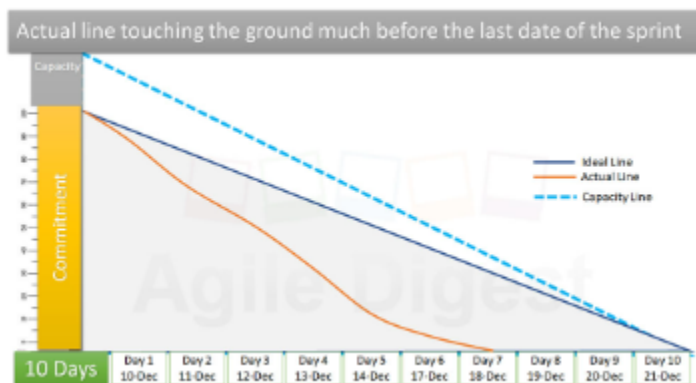
There are various trends in sprint burndown charts that can indicate unusual patterns in the actual progress line. Here are a few common irregular cases, their causes, and potential corrective actions.

- **Straight Actual Line**
- **Actual line touching the ground much before the last date of the sprint**
- **The actual line have Sudden Spike**
- **Ideal line and capacity line have too much difference on day 1**
- **The actual line is outside the ideal line for consistent days and the difference in increasing**
- **Straight lines and Sudden drop of the actual line**

Straight Actual Line

- **Causes:**
 - The team is not updating the remaining task hour on a daily basis
 - The team has not created all the tasks, so not able to change the remaining hours
 - Task estimation was underestimated, so the remaining hour is not changing
 - user stories are blocked by some impediments and those are not resolved, so the team is unable to proceed

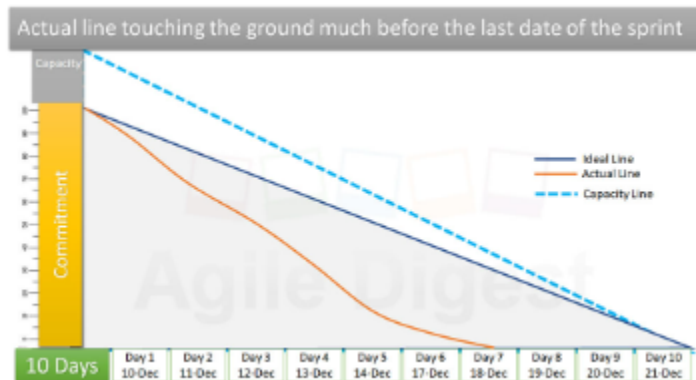
- An environmental issue like the development server not being available to do coding or the test environment not accessible for testing
- **Resolution:**
 - Ensure that every team member updates the remaining hours for their assigned tasks daily.
 - When breaking down a user story into tasks, take the time to carefully plan and estimate them accurately rather than rushing through the process.
 - Address impediments as quickly as possible. If a story is blocked, don't wait for its resolution—work on other available tasks. If all stories are blocked and the issue is expected to take a few days to resolve, escalate the matter to the Product Owner and take necessary actions.
 - Before starting the sprint, identify all dependencies and resolve them before committing to a story. If dependencies are unresolved, prioritize working on independent stories in the meantime.
 - Ensure that all environmental issues are addressed, and verify availability and accessibility before the sprint begins.



Actual line touching the ground

- **Causes**
 - The team has worked aggressively, burned more hours per day as per the capacity
 - The Team has committed less than the capacity
 - The team has overestimated the task.
- **Resolution.**

- Plan your capacity based on the best-optimized amount of work the team can perform.
- If the team commits less than the capacity, that happened because most of the time because of an insufficient amount of requirements, the Product Owner may not have enough story ready to commit at the time of planning. Work on frequent and regular grooming to have your backlog healthy.



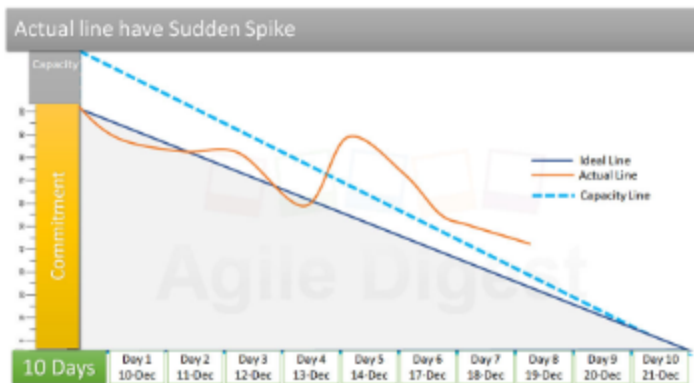
The actual line have Sudden Spike

- **Cause**

- change on scope (Scope Increased between sprint duration), Your PO may have asked to add one or few stories to the current sprint.
- The team has re-estimated the task to increase the remaining work.

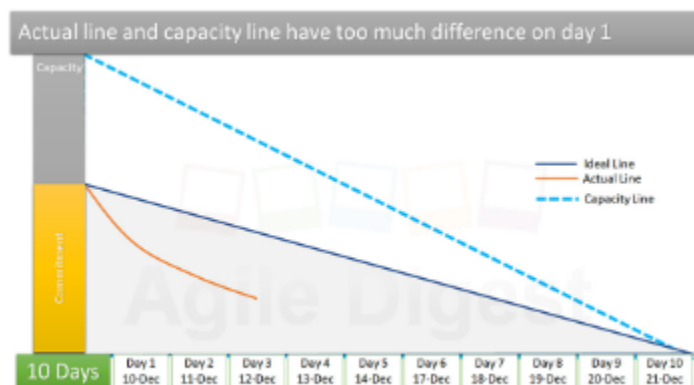
- **Resolution.**

- Mature up your Sprint Planning, Have your backlog healthy so that you can fully utilize your capacity during sprint planning, by committing max possible product backlog item in priority.
- Do a task breakdown thoroughly, and create all the possible tasks created, estimated, and assigned. so that you don't need to re-estimate during sprint duration.



Ideal line and capacity line have too much difference on day 1

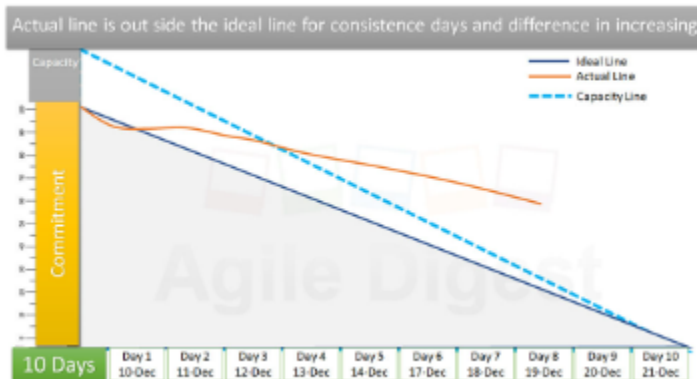
- **Cause**
 - Unavailability of enough user stories in ready state
- **Resolution**
 - Continuous backlog grooming to have a healthy backlog of ready work on Items.



The actual line is outside the ideal line for consistent days and the difference in increasing

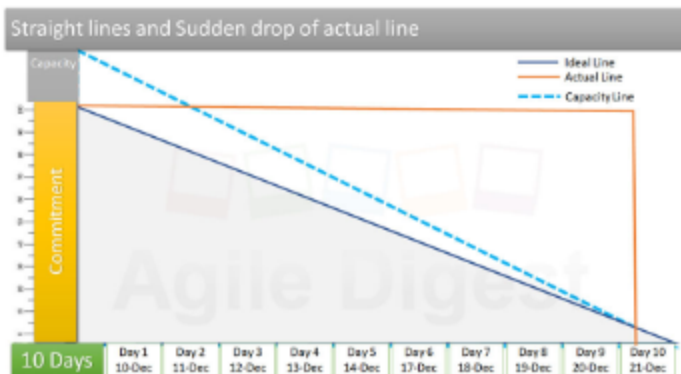
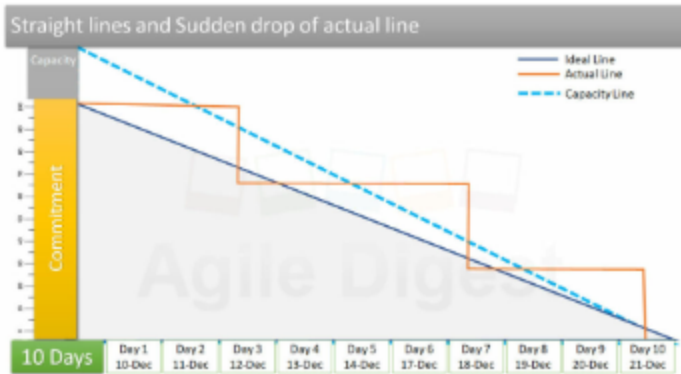
- **Cause**

- Even the team is working as expected, but they are not updating the remaining hours.
- The Team is actually not working
- The Team is blocked or impeded.
- The team has found estimated task hours are too low, and needs to work more, instead of re-estimating the task, they are not updating the remaining hours, working to bring the remaining hours as per burn down.
- **Resolution.**
 - Please update the remaining hours every day, it may bring some spike, and it will depict the real picture.
 - The rest of the resolution of the case varies from team to team, analyze it and take a best suited corrective action.



Straight lines and Sudden drop of the actual line

- **Cause.**
 - Remaining work updates are not happening, regularly.
 - Remaining work updates happen only on the last date of the sprint as a formality.
- **Resolution**
 - Start task breakdown, and have the Y-axis of your burndown with task hours
 - Update Remaining hours on daily basis.



UI - Investment Profile Requirements

D

By DEVQA

See views

Add a reaction



UI REQUIREMENT DOCUMENT

Feature: Release → Investment Profile

Platform: Web Application

Module: Release Management

1. OVERVIEW

The **Investment Profile** tab provides a consolidated analytical view of:

- Delivery metrics (planned vs completed)
- Work type investment distribution
- Cumulative sprint effort
- Sprint-wise effort breakdown (expandable)
- Budget vs actual analysis
- Team effort metrics (FTE)

This page is rendered for a **selected Release only** (single release context).

2. PAGE STRUCTURE

Layout Hierarchy

1. Header (Existing – No Change)

2. Delivery Summary (Full Width)

3. Row:

Left (50%) – Work Type Investment

Right (50%) – Cumulative Sprint Effort

4. Team Metrics Row (3 cards)

5. Budget & Cost Analysis (Full Width)

3. SECTION DETAILS

3.1 DELIVERY SUMMARY (FULL WIDTH)

Layout

Four summary cards aligned horizontally:

- Planned
- Unplanned
- Completed
- Not Completed

Data Fields

- Planned Count
- Unplanned Count
- Completed Count
- Not Completed Count

Visual Rules

- Planned → Neutral / Blue border
- Unplanned → Yellow border
- Completed → Green border
- Not Completed → Red border

This section spans entire page width.

3.2 WORK TYPE INVESTMENT (LEFT – 50%)

Component

Donut Chart

Data Segmentation

- Stories
- Bugs
- Tasks
- (Epics optional)

REQUIRED COLOR MAPPING (STRICT)

- **Stories → Green**
- **Bugs → Red**
- **Tasks → Blue**
- **Epics → Purple (optional default)**

This must be consistent across:

- Donut chart
- Stacked bars
- Budget bars
- Sprint breakdown
- Legends

Display Below Donut

For each work type show:

- Item count
- Total hours
- Total cost (\$)
- Percentage of total investment

3.3 CUMULATIVE SPRINT EFFORT (RIGHT – 50%)

IMPORTANT CHANGE

This section must show:

ONLY ONE STACKED VERTICAL BAR

It represents cumulative effort of all completed sprints combined.

Bar Segmentation

- Stories (Green)
- Bugs (Red)
- Tasks (Blue)

Displayed Values

Below bar show:

- Total Hours (e.g., 6,715h)
- Total Cost (e.g., \$197.5k)

Layout Notes

- Section height should visually match Work Type Investment.
 - Do not show multiple sprint cards here.
 - This is not sprint-by-sprint — this is cumulative only.
-

3.4 EXPANDABLE SPRINT BREAKDOWN

Below cumulative section:

Add toggle:

“View Sprint-wise Breakdown”

Behavior

- Default → Collapsed
- On Click → Expand new section below

Expanded Content

Display sprint-by-sprint stacked vertical bars.

Each sprint card must show:

- Sprint Name
- Stacked bar (Green/Red/Blue segments)
- Total Hours
- Total Cost (\$)

If the number of sprints exceeds available horizontal space, allow **horizontal scrolling** within this expanded section.

Hover / Tooltip Behavior

Each stacked segment must support hover interactions:

- When user hovers over the **Stories (Green)** segment, show a tooltip containing:
 - Work Type: Stories
 - Hours: <hours_for_stories_in_this_sprint>
 - Cost: \$<cost_for_stories_in_this_sprint>
- When user hovers over the **Bugs (Red)** segment, show a tooltip containing:
 - Work Type: Bugs
 - Hours: <hours_for_bugs_in_this_sprint>
 - Cost: \$<cost_for_bugs_in_this_sprint>
- When user hovers over the **Tasks (Blue)** segment, show a tooltip containing:
 - Work Type: Tasks
 - Hours: <hours_for_tasks_in_this_sprint>
 - Cost: \$<cost_for_tasks_in_this_sprint>

Tooltip should be lightweight, near cursor, and not obstruct the bar visibility.

3.5 TEAM METRICS ROW

Three horizontal cards:

1. Total Contributors

- Show numeric value only

2. Total Hours Logged

- Show total hours
- Optional subtext: % of capacity

3. FTE Equivalent

- Show FTE number
- Progress bar below

No growth indicators required.

3.6 BUDGET & COST ANALYSIS

Budget Summary (Left)

Display:

- Allocated Budget
 - Actual Spend
 - Variance (\$)
 - Variance (%)
 - Status badge:
 - Green → Under Budget
 - Red → Over Budget
-

Budget vs Actual by Work Type (Right)

Horizontal comparison bars:

For each:

- Stories
- Bugs
- Tasks

Bar 1 → Actual Spend

Bar 2 → Allocated Budget

Use consistent color mapping:

- Stories → Green
 - Bugs → Red
 - Tasks → Blue
-

4. DATA DEFINITIONS (FOR UI BINDING)

Planned → Items committed before sprint start

Unplanned → Items added after sprint start

Completed → Items completed within release

Not Completed → Items carried forward

Cumulative Sprint Effort:

Sum of all sprint hours aggregated.

Cost Calculation:

Sum (Actual Hours × Resource Hourly Rate)

FTE: Total Hours ÷ Standard capacity hours

5. COLOR SYSTEM

Work Type	Color
Stories	Green
Bugs	Red
Tasks	Blue
Epics	Purple

These must be consistent across entire page.

6. RESPONSIVENESS

This implementation is for **web only**.

- Use equal width sections for Investment & Cumulative.
- Maintain alignment and spacing consistency.
- Use consistent padding across cards.

7. NON-FUNCTIONAL REQUIREMENTS

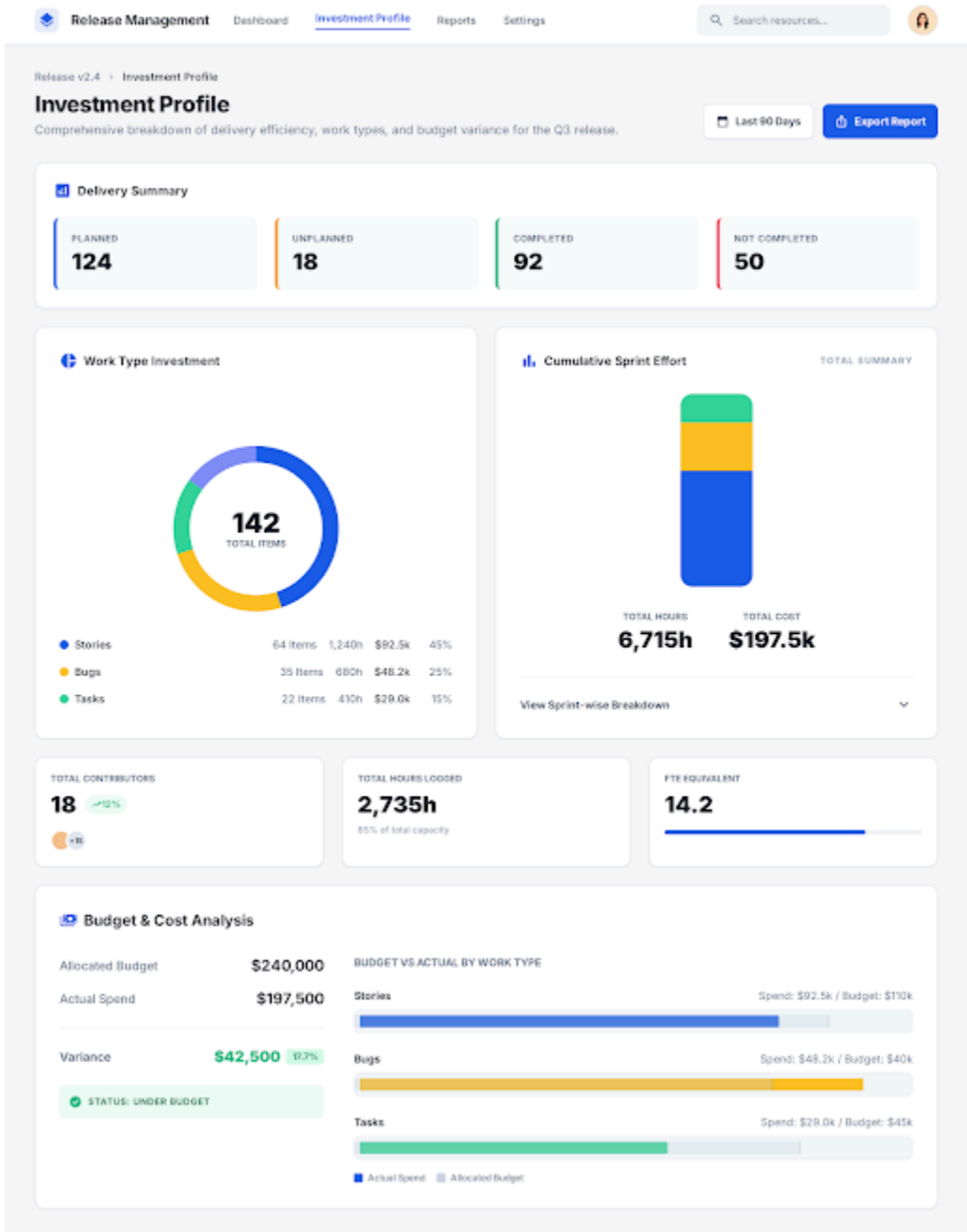
- Clean SaaS UI
- Card-based layout
- Subtle shadows
- Consistent spacing
- No animated trend indicators
- No percentage growth badges
- No extra arrows/icons not defined in spec

8. FINAL VALIDATION CHECKLIST FOR UI DEV

Before marking complete:

- Delivery spans full width
- Investment & Cumulative sections equal width
- Only one cumulative stacked bar
- Sprint breakdown expandable
- Story = Green everywhere
- Bug = Red everywhere
- Task = Blue everywhere
- Budget section

Design/Screen Shot from AI Tool



Release Burndown

Product Requirement Document: Release Burndown Chart for Jira

1. Overview

1.1 Purpose

This document defines the requirements for a **Release Burndown Chart for QMetry360**. The feature will use **Jira issues, Fix Versions, Sprints, and Story Points** to visualize progress toward completing the scope of a Jira **Release** (i.e., Fix Version) over time.

1.2 Background & Context

Jira teams typically track release scope with:

- **Projects**
- **Fix Versions / Releases**
- **Issue Types** (Story, Bug, Task, etc.)
- **Story Points / Time Estimates**
- **Boards & Sprints**

While Jira already provides sprint-level reports, teams often lack a **clear, version-centric view** of progress across multiple sprints. A **Release Burndown Chart** gives this by focusing on a Fix Version and:

- Showing remaining work (e.g., story points) vs time
- Visualizing scope changes (issues added/removed from the version)
- Highlighting risk of missing the release date

2. Objectives & Goals

2.1 Primary Objectives

1. **Release-level visibility**
 - Provide a Release Burndown report for **any Jira Fix Version** across one or more Jira projects.
2. **Data-driven release management**
 - Help Product Owners and Project Leads answer:
 - “Are we on track to complete this Fix Version by its Release Date?”
 - “What is the impact of issues being added or removed mid-release?”
3. **Transparent scope changes**
 - Highlight when the **Fix Version contents change** (scope creep/shrink) and how that affects burndown.

2.2 Non-Goals

- Replace Jira’s existing **Sprint Burndown / Velocity** reports.
- Provide team-member level performance analytics.
- Implement complex probabilistic forecasting beyond a simple trend-based projection.

3. Scope

3.1 In Scope

- **Release Burndown** report view inside QMetry360, driven by:
 - **Fix Version** (primary)
 - Optional filters: Project, Board, Issue Type, JQL
- Use **Story Points** as the primary metric; support:
 - (Optional) Remaining Time Estimate as a secondary metric
- Chart:
 - Actual remaining work over time
 - Ideal burndown line from initial scope to 0 on target release date
 - Visual indication of scope change (issues added/removed from Fix Version)
 - Simple forecast of completion date based on recent throughput (optional)

3.2 Out of Scope (initial release)

- Multi-version comparative charts (e.g., “compare Release 1.0 vs 2.0” on the same graph).
- Cross-Jira-instance aggregation.
- ML-based forecasting.

4. Users & Use Cases

4.1 Target Users

- **Product Owners / Project Leads**
 - Want to see if the Fix Version is likely to ship on time and adjust scope.
- **Scrum Masters**
 - Use the chart to facilitate release planning and review during ceremonies.
- **Engineering Managers**
 - Track release risk across teams/boards.
- **Business Stakeholders**
 - Need high-level progress and risk view for a given release.

4.2 Key Use Cases

1. **Track Fix Version progress across sprints**
 - PO selects a Fix Version (e.g., QM 1.5.0) and sees remaining Story Points vs the Version Release Date.
 2. **Visualize scope creep via Fix Version changes**
 - Issues are added to the Fix Version mid-release.
 - Burndown shows sudden increases with annotated markers.
 3. **Use in Sprint Reviews / Release Planning**
 - During sprint review, team checks whether the current burndown trend supports the planned Release Date and adjusts Fix Version scope accordingly.
 4. **Post-release retrospective**
 - Team reviews how many issues were added/removed from the Fix Version and how that affected burndown and timeline.
-

5. Functional Requirements

5.1 Jira Integration & Scope Definition

FR-1: Select Release (Fix Version)

- User can open a “**Release Burndown**” report in QMetry360 and:
 - Select a **Project**.
 - Select a **Fix Version**
- The Release entity is defined by:
 - Jira Fix Version fields:
 - Name (e.g., Release 1.2.0)
 - Start date (derived or manually configured)
 - **Release Date** (from Fix Version, if set) – used as target date.

FR-2: Configure Start Date

- Start date for the chart can be:
 - (Default) Earliest of:
 - First issue added to Fix Version
 - Start date of the first sprint containing issues in this Fix Version
 - (Optional) Manually overridden in the report settings.

FR-3: Associate Jira Issues with Release

- Issues included in the burndown:
 - All issues where:
 - `fixVersion` contains the selected Fix Version, and
 - (Optional) match additional filters:
 - Project(s)
 - Board
 - Issue Type(s)
 - Custom JQL (e.g., exclude subtasks, or filter specific labels)
- Changes to an issue's `fixVersion` are treated as scope changes.

5.2 Measurement Units

FR-4: Metric Selection

- Primary metric: **Story Points** (from Jira Story Points field; configurable field name).
- Fallback metric:
 - If Story Points are missing for an issue, use 1 or a configured default per issue.
 - Alternatively, user can choose:
 - **Number of Issues** as metric.
 - (Optional) Support Remaining Estimate (time) as metric.

5.3 Chart Visualization

FR-5: Burndown Chart Rendering

- X-axis:
 - Time-based; options:
 - ~~Per day between start date and Release Date.~~
 - **Per sprint**, if a board is selected and issues are assigned to sprints.
- Y-axis:
 - Remaining work in the chosen metric (Story Points or issue count).
- Lines:
 - **Actual Remaining Line**: Remaining metric at each time interval.
 - **Ideal Line**: Linear burn from initial scope at start date to zero at Release Date.
- Optional:
 - Data points with hover tooltips.

FR-6: Scope Change Visualization

- When issues are added/removed to/from the Fix Version:
 - The total scope line adjusts from that date forward.
- Visual indicators:
 - Markers on the chart:
 - “+X pts / +N issues added to release”
 - “-Y pts / -M issues removed from release”
- (Optional) Side panel listing scope change events with:
 - Date
 - JQL link to affected issues.

FR-7: Time Granularity Controls

- Users can choose:
 - **Daily view**
 - **Sprint view** (if a board is selected and data is available)
- Default:
 - If a board is selected → Sprint-based.
 - ~~Else → Daily-based.~~

5.4 Interactions & Filters

FR-8: Filters & Configuration

In the report UI, user can configure:

- Project(s) (multi-select)
- Board (optional; for sprint-based aggregation)
- Fix Version (required)
- Metric:
 - Story Points
 - Number of Issues
- Date range:
 - Default: from start date to Release Date
 - Optional custom: from start date to “today” or custom end date
- Toggles:
 - Show/Hide ideal line
 - Show/Hide scope change markers
 - Show/Hide forecast line (if enabled)

FR-9: Tooltips & Details on Hover

- Hover over any data point shows:
 - Date or Sprint name
 - Remaining work
 - Completed work to date
 - Total scope at that point
 - Scope change note if relevant on that day

FR-10: Forecast / Completion Projection (MVP+)

- Optional simple forecast:
 - Compute average burn rate based on last N intervals (configurable, default 3–5).
 - Draw a projection line to estimate when remaining work would reach 0.
- Display:
 - “Estimated completion: [date] at current average burn rate.”
 - If projected date > Fix Version Release Date:
 - Show a risk indicator (e.g., warning icon + message: “At current pace, release may slip to [date].”).

5.5 States & Edge Cases (Jira-Specific)

FR-11: Before Work Starts

- If no issues in the Fix Version are in “In Progress” or “Done”:
 - Actual remaining line is flat at initial total scope.
 - Note: “No completed issues yet for this release.”

FR-12: After Release Date with Remaining Scope

- If current date > Fix Version Release Date and remaining work > 0:
 - Extend actual line beyond Release Date.
 - Ideal line ends at Release Date.
 - Display message: “Release date passed; X [points/issues] still open.”

FR-13: Completed Release

- If all issues in Fix Version are in a “Done” status:
 - Actual line reaches 0 on the completion date.
 - Optional annotation: “Release scope completed on [date].”

FR-14: No Data / Misconfiguration

- If the selected Fix Version has:
 - No issues, or
 - Only issues without the selected metric (e.g., no Story Points and metric is Story Points):
 - Show an empty state:
 - “No data to display. Ensure issues with this Fix Version have [Story Points / metric] and are in the selected projects/filters.”
-

6. Non-Functional Requirements

6.1 Performance

- For each chart load:
 - Handle up to:
 - 10,000 issues in the selected Fix Version(s)
 - 2–3 years of history
 - Target response time:
 - < 3 seconds for typical usage
 - < 5 seconds for maximum scope

6.2 Security & Permissions

- Respect Jira permissions:
 - Only include issues the current user is allowed to see.
 - If user lacks access to some issues in the Fix Version:
 - Either:
 - Exclude them from calculations, or
 - Show a note: “Some issues are hidden due to permissions; data may be incomplete.”
- The report should be visible only to users with permission to view the underlying projects.

6.3 Usability & Accessibility

- Use Jira-like visual style and patterns.

- Provide:
 - Legend explaining lines & markers.
 - Color-blind-friendly palette.
 - Support keyboard navigation:
 - For report filters and actions.
-

7. Data Model & Calculations (Within Jira Context)

7.1 Inputs

From Jira:

- **Fix Version**
 - `id`, `name`, `releaseDate` (optional), `project(s)`, `description`
- **Issues**
 - `id`, `key`
 - `project`
 - `fixVersion(s)`
 - `issueType`
 - `status` and `status category` (To Do / In Progress / Done)
 - `Story Points` field (configurable)
 - `history`:
 - Changes to status (with timestamps)
 - Changes to Fix Version (with timestamps)
 - (Optional) Sprint membership (for sprint-based aggregation)

7.2 Remaining Work per Interval

For each time interval t (day or sprint):

- `totalScope(t)`:
 - Sum of metric (Story Points or 1/issue) for all issues that:
 - Had the selected Fix Version assigned at any point on or before t .
- `completedWork(t)`:
 - Sum of metric for issues in a **Done** status at time t .

- $\text{remainingWork}(t) = \text{totalScope}(t) - \text{completedWork}(t).$

7.3 Ideal Line

- Let:
 - S_0 = total scope at the start date
 - D = number of intervals between start date and Release Date (or end date if Release Date is blank)
 - For interval index i ($0..D$):
 - $\text{idealRemaining}(i) = S_0 * (1 - i / D).$
-

8. Jira UX & UI Requirements

8.1 Navigation

- Report entry points:
 - Left-side **Reports** section in a Jira Software project (new “Release Burndown” report).
 - (Optional) **Version details** screen: link to “View Release Burndown”.
 - (Optional) Global app menu if this is a Marketplace app.

8.2 Layout

- Top: Filter controls (Project, Board, Fix Version, Metric, Date Range, toggles).
- Main area: Chart with:
 - Legend:
 - Actual Remaining
 - Ideal Burndown
 - Scope Changes
 - Forecast (if enabled)
- Right or bottom panel:
 - Key stats:
 - Total scope
 - Completed scope
 - Remaining scope
 - Days until Release Date
 - Forecast completion date (if enabled)

- Scope change event list (optional).
-

9. Analytics & Tracking

- Track report usage:
 - Number of views per Fix Version
 - Common filters / metrics chosen
 - Time spent on the report
 - Track errors:
 - Data retrieval failures from Jira
 - Chart rendering issues
-

10. Risks & Limitations (Jira-Specific)

- **Lack of Story Points:**
 - Many projects may not use Story Points consistently; fallback metric is # of issues.
 - **History & Performance:**
 - Reconstructing historical scope based on issue change history (especially Fix Version changes) could be expensive; need caching.
 - **Multiple Fix Versions per issue:**
 - Decide behavior:
 - Count the issue once if it contains the selected Fix Version, regardless of others.
 - **Cross-project releases:**
 - Different workflows and status categories can complicate “Done” semantics; require mapping to Jira status category = “Done”.
-

11. Dependencies

- Jira Software:
 - Access to:
 - Issues and their changelogs (for Fix Version & status changes)
 - Fix Versions and Release Date

- Boards and Sprints (if sprint-based view is used)
 - Charting library compatible with Jira UI.
-

12. Sample Acceptance Criteria (Jira-Flavored)

1. Basic Burndown Rendering

- Given a Fix Version with issues that have Story Points
When a user opens the Release Burndown report and selects that Fix Version
Then the chart shows:
 - An actual remaining line over time
 - An ideal line from start date to Release Date.

2. Fix Version Scope Change

- Given issues are added to the selected Fix Version after the start date
When the report is refreshed
Then the chart shows an increase in total remaining work from that date
And a scope change marker is visible with total added points.

3. Forecast vs Release Date

- Given the average burn rate predicts completion after the Fix Version Release Date
When the forecast is enabled
Then the chart shows a forecast completion date later than the Release Date
And a warning indicator is displayed.

4. Permissions

- Given a user without access to some issues in the Fix Version
When the user opens the report
Then only accessible issues are included in the burndown
And a note is displayed that some issues are hidden due to permissions.
-

13. Release Forecast & Burn-up Dashboard (Enhanced View)

13.1 Overview

This section defines additional requirements for an enhanced **Release Forecast & Burn-up Dashboard** view within QMetry360.

The objective of this enhancement is to provide:

- Predictive release delivery insights
- Real-time release health indicators
- Risk and alert visibility
- Regression testing visibility
- Bug priority distribution analytics

This dashboard supplements the Release Burndown report but does not replace the existing Velocity, Sprint Burndown, or Backlog Health reports.

13.2 Release Status Indicator

FR-15: Release Health Label

The system shall display a release health status indicator on the **top-left corner** of the page.

Possible statuses:

-  On Track
-  At Risk
-  Off Track

Status Determination Logic

The status shall be calculated based on:

- Forecasted completion date vs Release Date
- Confidence score threshold
- Open critical issues
- Recent velocity trend

Rules (MVP logic):

- If forecast date $\leq$ Release Date AND confidence $\geq 75\%$ → **On Track**
 - If forecast date $>$ Release Date by ≤ 1 sprint OR confidence between 60–74% → **At Risk**
 - If forecast date $>$ Release Date by > 1 sprint OR confidence $< 60\%$ → **Off Track**
-

13.3 Release Summary Metrics Panel (Top-Right Section)

FR–16: Summary KPI Cards

The dashboard shall display summary KPI cards on the top-right section including:

1. **Confidence Score (%)**
2. **Total Scope (Story Points / Issues)**
3. **Open Issues**
4. **Critical Issues Count**

Confidence Score Calculation (MVP)

Confidence score shall be derived from:

- Variance between ideal and actual burn
- Stability of velocity (last 3–5 sprints)
- Scope change frequency
- Critical open defects

The formula may be heuristic-based for MVP and refined in later versions.

Tooltip shall explain contributing factors.

13.4 Release Forecast & Burn-up Chart

FR–17: Burn-up Style Visualization

The main chart shall display:

X-axis: Time (Sprint-based preferred; daily optional)

Y-axis: Completed Scope vs Total Scope

The chart shall include:

1. Actual progress line (solid)
 2. Ideal projection line
 3. Forecast projection lines (dotted)
-

FR–18: Remaining Sprints & Projected Delivery Date

Below or inside the chart area, the system shall display:

- **Remaining Sprints** (based on forecast vs current sprint velocity)
- **Projected Delivery Date**

Example display:

- Remaining Sprints: 4
- Projected Delivery: Oct 30, 2023

Calculation:

Remaining Sprints = Remaining Scope ÷ Average Velocity (last N sprints)

FR–19: Multiple Forecast Projections (Dotted Lines)

The chart shall support three projection scenarios:

1. **Likely (Default)**
Based on average velocity of last 3–5 sprints.
2. **Pessimistic**
Based on lowest velocity in last N sprints.
3. **Optimistic**
Based on highest velocity in last N sprints.

Each projection shall be represented as a dotted line.

Legend shall clearly distinguish:

- Likely
- Pessimistic
- Optimistic

User shall have toggle options to show/hide projections.

13.5 Risk & Alerts Panel

FR–20: Risk & Alerts Section (Right Panel)

The dashboard shall include a dedicated **Risk & Alerts** panel on the right side displaying system-generated alerts.

Alert categories:

1. **Blockers Detected**
 - If issues in “Blocked” status > threshold

- Display count and severity
- Provide link to filtered issue list
- 2. **Velocity Drop**
 - If velocity dropped > 20% compared to last sprint average
 - Display comparison insight
- 3. **Scope Update**
 - If scope increased > 10% after start date
 - Show total points added

Each alert shall:

- Show severity level (High / Medium / Low)
 - Be clickable for drill-down
 - Be time-stamped
-

13.6 Regression Testing Summary

FR-21: Regression Testing Status Display

The dashboard shall display a regression testing summary section including:

- Total regression test cases
- Passed %
- Failed %
- Blocked %

Visual representation:

- Progress bar or summary card
- Highlight failure rate if > threshold (e.g., 15%)

If integrated with QMetry test management:

- Pull execution results linked to Fix Version
 - Use latest execution cycle tagged to the release
-

13.7 Bug Priority Distribution

FR-22: Priority Distribution Visualization

The dashboard shall display a bug priority distribution chart showing:

- High Priority %
- Medium Priority %
- Low Priority %

Visualization:

- Pie chart or donut chart
- Color-coded legend
- Hover tooltip with exact counts

Data Source:

All open bugs in selected Fix Version.

Qmetry Roadmap (March – April 2026)

Qmetry AI-Driven Delivery Platform Roadmap

Immediate → End of April 2026

 AI-DLC Implementation  AI-DLC Metrics  RBAC  Milestone  = Deliverable

Now → Apr Wk1

Apr Wk2 → Apr 15

Mid April

End April

1. AI-DLC Implementation

Milestone 1 — AI-DLC Understanding & Readiness

- 🕒 Week 1 · End by ~April 1st week
- Review Trigent ANDIS AI-DLC framework
- Map Qmetry SDLC → AI-DLC stages
- Identify gaps, integrations & data readiness

◆ Alignment Doc ◆ Readiness Assessment ◆ Tooling Plan

Milestone 2 — AI-DLC Implementation

- 🕒 Apr Wk2 → April 15
- Configure AI-DLC aligned workflows
- AI-assisted backlog, QA & sprint exec
- AI-driven dashboards & automation

◆ AI-DLC sprint execution ◆ Usage guidelines

📊 Success: % AI-DLC adoption in sprints · Target completion: April 15

2. AI-DLC Metrics Implementation

Milestone 1 — Metrics Definition

- 🕒 Early–Mid April
- Analyze current SDLC metrics
- Define AI-DLC aligned metrics:
AI dev coverage, automation %, defect prediction, lead time reduction

◆ Metrics Definition Doc ◆ Metrics Mapping
◆ JIRA Epics & Stories

Milestone 2 — Implement

- 🕒 Mid–Late April
- Data model updates
- Dashboard enhancements
- Metric compute logic
- AI vs non-AI comparison
- Trend & predictive views

Milestone 3 — Deploy

- 🕒 End of April
- Validate accuracy
- Data consistency
- Dashboard QA
- Production deploy

◆ Metrics live
◆ Docs

📊 Success: AI-DLC metrics live in Qmetry dashboards · Target: End of April

3. Role-Based Access Control (RBAC)

Milestone 1 — Requirements & Design

- 🕒 Upcoming week
- Define roles: Admin, PM, Dev, Stakeholder
- Define access: project, module, action level
- Stakeholder discussions

◆ RBAC Requirement Document
◆ Role-Permission Matrix

Milestone 2 — Implementation

- 🕒 Mid-April
- Role assignment framework
- Permission enforcement
- UI: restricted views & role mgmt

Milestone 3 — Validate

- 🕒 End of April
- Functional validation
- Access verification

◆ RBAC-enabled
◆ Validation report

📊 Success: Controlled access · Zero unauthorized changes · Improved governance

Objective

Enable Qmetry as a governed, AI-driven delivery platform

- AI-DLC Implementation (complete by Apr 15)
- AI-DLC Aligned Metrics (complete by end April)
- Role-Based Access Control / RBAC (complete by end April)

Objective

Enable Qmetry as a **governed, AI-driven delivery platform** by:

- Implementing **AI-DLC (AI-driven Delivery Lifecycle)**
 - Introducing **AI-DLC aligned Metrics**
 - Enabling **Role-Based Access Control (RBAC)**
-

1. AI-DLC Implementation in Qmetry (Aligned with Trigent Framework)



Timeline

Start: Immediate

Target Completion: 15th April 2026

◆ Milestone 1: AI-DLC Understanding & Readiness

Timeline: Week 1 (End by ~April 1st week)

Key Activities

- Review **Trigent ANDIS AI-DLC framework**
- Map Qmetry's current SDLC workflows to AI-DLC stages:
 - Planning
 - Development
 - QA
 - Release
 - Feedback loop
- Identify:
 - Gaps in workflows
 - Required AI/tool integrations
 - Data readiness (velocity, defects, automation, etc.)

Deliverables

-  AI-DLC Alignment Document
 -  Clarifications & open questions
 -  Tooling & Licensing Plan
 -  AI-DLC Readiness Assessment
-

◆ Milestone 2: AI-DLC Implementation

Timeline: April 2nd week → **April 15th**

Key Activities

- Configure Qmetry workflows aligned to AI-DLC:
 - AI-assisted backlog refinement
 - AI-assisted QA & automation integration
 - Sprint execution aligned to AI-DLC
- Enable:
 - AI-driven dashboards (forecast, risk, quality)
 - Automation within sprint lifecycle

Deliverables

-  AI-DLC enabled sprint execution
 -  Team usage guidelines
-

Dependencies

- Tool/license approvals
 - AI integration feasibility
 - Team readiness
-

Success Metrics

- % of AI-DLC adoption in sprints



2. AI-DLC Metrics Implementation in Qmetry



Timeline

Start: Immediate

Target Completion: End of April 2026

◆ Milestone 1: Metrics Definition & Alignment

Timeline: Early–Mid April

Key Activities

- Analyze **current SDLC metrics in Qmetry**
- Identify:
 - Metrics to **retain**
 - Metrics to **enhance**
 - Metrics to **deprecate**
- Define **AI-DLC aligned metrics**, such as:

- AI-assisted development coverage (%)
- Automation within sprint (%)
- Defect prediction accuracy
- Test case auto-generation coverage
- Lead time reduction (AI vs non-AI)
- Release predictability (AI forecast vs actual)
- Conduct stakeholder workshops

Deliverables

-  AI-DLC Metrics Definition Document
 -  Metrics Mapping (SDLC → AI-DLC)
 -  JIRA Epics & Stories for implementation
-

◆ Milestone 2: Implementation

Timeline: Mid–Late April

Key Activities

- Implement new metrics in Qmetry:
 - Data model updates
 - Dashboard enhancements
 - Metric computation logic
- Enable visualization:
 - AI vs non-AI comparison

- Trend tracking
 - Predictive insights
-

◆ Milestone 3: Testing & Deployment

Timeline: End of April

Key Activities

- Validate:
 - Metric accuracy
 - Data consistency
 - Dashboard correctness
- Deploy to production

Deliverables

-  AI-DLC Metrics available in Qmetry
 -  AI metrics dashboards
 -  Documentation for interpretation
-



3. Role-Based Access Control (RBAC) in Qmetry



Timeline

Start: Immediate

Target Completion: End of April 2026

◆ Milestone 1: Requirement Gathering & Design

Timeline: Upcoming Week

Key Activities

- Define user roles:
 - Admin, PM, Developer, Stakeholder
- Define access levels:
 - Project-level
 - Module-level
 - Action-level
- Conduct stakeholder discussions

Deliverables

-  RBAC Requirement Document
 -  Role-Permission Matrix
-

◆ Milestone 2: Implementation

Timeline: Mid-April

Key Activities

- Implement:
 - Role assignment framework
 - Permission enforcement
 - UI updates:
 - Restricted views
 - Role management
-

◆ Milestone 3: Testing & Validation

Timeline: End of April

Key Activities

- Functional validation
- Access verification

Deliverables

-  RBAC-enabled Qmetry
 -  Validation report
-



Success Metrics

- Controlled access enforcement
 - Zero unauthorized changes
 - Improved governance
-



Consolidated Timeline View

Initiative	Milestone	Timeline	Status
AI-DLC	Understanding & Planning	Week 1	
AI-DLC	Implementation	By Apr 15	
AI-DLC Metrics	Definition	Early–Mid Apr	

AI-DLC Metrics	Implementation	Mid–Late Apr	
AI-DLC Metrics	Testing & Deployment	End Apr	
RBAC	Requirement Gathering	Upcoming Week	
RBAC	Implementation	Mid Apr	
RBAC	Testing	End Apr	

Initiative	Milestone	Timeline	Status
AI-DLC	Understanding & Planning	Week 1	
AI-DLC	Implementation	By Apr 15	

AI-DLC Metrics	Definition	Early–Mid Apr	🕒
AI-DLC Metrics	Implementation	Mid–Late Apr	🕒
AI-DLC Metrics	Testing & Deployment	End Apr	🕒
RBAC	Requirement Gathering	Upcoming Week	🔄
RBAC	Implementation	Mid Apr	🕒
RBAC	Testing	End Apr	🕒



Key Risks & Mitigation

Risk	Impact	Mitigation

Undefined AI metrics	Delay in implementation	Early workshops + alignment
Tool dependency delays	AI-DLC impact	Parallel evaluation
RBAC complexity	Delay	MVP-first approach
Data inconsistency	Incorrect AI insights	Data validation layer

SDLC Industry Standards

The SDLC process outlined in this document has been designed in alignment with widely adopted industry standards and market best practices followed across modern Agile and DevOps-driven organizations. The process emphasizes end-to-end traceability, structured requirement management, collaborative development, peer code reviews, quality assurance, automation, security validation, and controlled release management to ensure scalability, maintainability, and delivery excellence.

The framework also incorporates key governance practices such as workflow standardization, effort tracking, audit readiness, defect management, CI/CD integration, and measurable quality metrics to support transparency and operational efficiency across the software delivery lifecycle. The objective is to establish a consistent, scalable, and compliance-oriented engineering process that improves predictability, accelerates delivery, and enhances overall product quality.

Tickets Effort Tracking and Worklow

Add a reaction

Jira JQL has **very limited support for querying worklogs**. You cannot efficiently do advanced queries like:

- “Show tickets where tester logged > 4h”
- “Find tickets without reviewer effort”
- “Developer effort vs QA effort”
- “Worklogs by role”
- “Reviewer efficiency”

using native Jira alone.

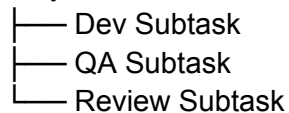
Industry Approaches to Solve This

Option 1 — Use Subtasks (Most Common)

Instead of relying purely on worklogs:

Structure:

Story



Now Jira queries become easy:

Examples:

issuetype = Sub-task AND summary ~ "QA"

or:

assignee = tester1 AND issuetype = Sub-task

👉 This is why many enterprises prefer subtasks over worklog analytics.

Option 2 — Use Components / Labels / Custom Fields

Example:

- Work Type = Development
- Work Type = Testing
- Work Type = Review

Then query becomes easier.

But:

✗ still weak for effort analytics.

Option 3 — Use Jira Plugins (Most Enterprise Teams)

Most mature organizations use plugins like:

- Tempo Timesheets
- eazyBI
- ScriptRunner

These provide:

- Worklog queries
- Role-based analytics
- Efficiency dashboards
- Capacity reports



What Most Mature Teams Actually Do

Agile Product Teams

Usually:

- Minimal worklog dependency

- Focus on story completion
- Use subtasks only when needed

Enterprise / Service / Audit-heavy Teams

Usually:

- Mandatory worklogs
- Role-based subtasks
- Tempo/eazyBI dashboards



Recommended Practical Model

BEST BALANCE

Parent Story

Tracks:

- business delivery
- velocity

Subtasks

Tracks:

- Dev
- QA

- Review

Worklogs

Tracks:

- actual effort

Example Query Possibilities

Find all QA tasks

issuetype = Sub-task AND labels = QA

Find review tasks not completed

summary ~ "Review" AND status != Done

Find developer effort

Using:

- assignee
- subtasks
- Tempo reports



Important Reality

Trying to derive:

- developer productivity
- tester efficiency
- reviewer effectiveness

purely from Jira worklogs is usually:

✗ unreliable

because:

- logging discipline varies
- effort $\neq$ value delivered



Final Recommendation

If you need:

Basic Agile tracking

✓ Single story + optional subtasks

If you need:

Governance / utilization / audit reporting

✓ Use:

- subtasks
- mandatory worklogs

- Tempo/eazyBI



Practical Enterprise Standard

Need	Best Practice
Sprint velocity	Parent stories
Role effort tracking	Subtasks
Utilization reports	Tempo/eazyBI
Reviewer metrics	PR tools (Bitbucket/GitHub)
QA efficiency	Defect analytics

Sprint Churn

Sprint churn refers to:

The percentage of work added, removed, or significantly changed after the sprint has

started.

Typical churn includes:

- New stories added mid-sprint
- Scope changes
- Story removals
- Re-estimation causing major effort variation



Common Industry Benchmarks

Sprint Churn %	Industry Interpretation
0–5%	● Excellent / Stable Sprint
5–10%	● Acceptable / Manageable
10–20%	● High Churn / Planning Concern

>20%

● Unstable Sprint / Process Risk

Dashboard Module	Main APIs Identified
Standup Dashboard	<code>/api/analytics/getStandupData, /api/standup/*</code>
Jira Dashboard	<code>/api/jira/getVelocity, /api/jira/getCycleTime, /api/jira/getBurndownData</code>
Git Dashboard	<code>/api/github/*, /api/gitlab/*, /api/analytics/getGitData</code>
CXO Dashboard	<code>/api/cxo/*, /api/analytics/getCXOData</code>
Release Dashboard	<code>/api/releaseDashboard/releaseData</code>
Tech Quality Dashboard	<code>/api/techQuality/getTechQualityMetrics</code>